

CPU Scheduling Optimization and Process Monitoring System

Operating Systems Course Project

C++ Implementation with CMake Build System

C++17 Standard

Built with GCC 16.1.0 via MSYS2 on Windows

Target Platform: Linux (primary) / Windows (compatibility)

Project Location: C:\Users\foysa\cpu-scheduling-optimizer

Table of Contents

1. Introduction and Problem Statement
2. Project Objectives
3. System Architecture
4. Linux Process Monitoring via /proc
5. C++ System Calls and OS Interfaces
6. Data Models
7. Scheduling Algorithms
8. Performance Metrics
9. Optimization Engine
10. Workload Classification
11. Algorithm Test Results
12. Real System Test Results
13. Build Instructions
14. Limitations
15. Viva Preparation - Key OS Concepts

1 Introduction and Problem Statement

Operating systems use CPU scheduling algorithms to determine which process runs on the CPU at any given time. The choice of algorithm significantly impacts system performance metrics such as waiting time, turnaround time, response time, CPU utilization, and fairness.

This project implements a CPU Scheduling Optimization and Process Monitoring System that:

- Collects actual process information from the OS using C++ system calls
- Analyzes the real workload and classifies its characteristics
- Runs five classical CPU scheduling algorithms against the workload
- Calculates comprehensive performance metrics for each algorithm
- Normalizes and scores each algorithm using a weighted optimization model
- Recommends the best scheduling strategy with a dynamic explanation

IMPORTANT: This system monitors actual OS processes but does NOT replace or modify the Linux kernel CPU scheduler. The scheduling algorithms are used for analysis, comparison, and optimization only.

2 Project Objectives

- Demonstrate practical knowledge of system calls and OS interfaces
- Implement all five classical CPU scheduling algorithms
- Calculate and compare scheduling metrics (7 metrics)
- Build an optimization engine that scores and recommends the best algorithm
- Monitor real system processes safely (read-only)
- Provide a terminal UI for interaction
- Use C++17 with CMake build system
- Write comprehensive tests (35 test cases)

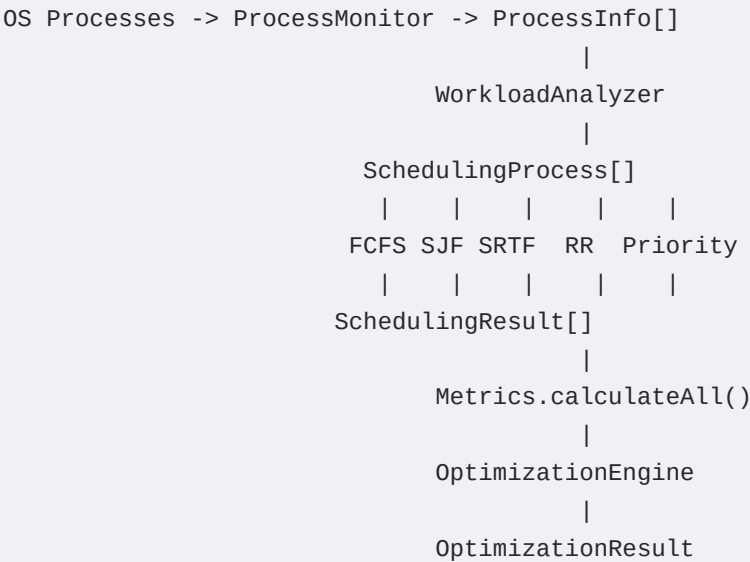
3 System Architecture

Module Overview

The system consists of six major modules:

- 1. ProcessMonitor - Collects real process information from the OS
- 2. Scheduling Algorithms (5 classes) - FCFS, SJF, SRTF, RR, Priority
- 3. Metrics Calculator - Computes 7 performance metrics
- 4. Workload Analyzer - Classifies the system workload type
- 5. Optimization Engine - Scores, ranks, and recommends algorithms
- 6. Terminal UI / GUI - User interface for interaction

Data Flow



File Structure (28 files)

include/	14 header files
src/	10 source files (incl. main.cpp)
tests/	1 test file (35 test cases)
docs/	1 architecture document
CMakeLists.txt	Build configuration

4 Linux Process Monitoring via /proc

Linux provides direct access to process information through the /proc virtual filesystem. This is a pseudo-filesystem that exists only in memory and provides an interface to kernel data structures.

Key /proc Files

File	Purpose
/proc/[pid]/stat	Process status: CPU times, priority, state
/proc/[pid]/status	Process memory and thread info
/proc/[pid]/cmdline	Command line arguments
/proc/stat	System-wide CPU statistics
/proc/meminfo	System memory information

Parsing /proc/[pid]/stat

The stat file contains a single line with space-separated fields. Fields 14 and 15 contain user and kernel CPU time in clock ticks. Field 18 is priority, field 19 is nice value, field 21 is start time. The process name is in parentheses.

Process Disappearance Handling

Processes can terminate during collection. The system handles this gracefully:

```
try {
    ProcessInfo info = parseLinuxProcStat(pid);
    if (info.pid > 0 && !info.name.empty())
        processes.push_back(info);
} catch (...) {
    continue; // Skip disappeared process
}
```

5 C++ System Calls and OS Interfaces

Linux Interfaces

Interface	Purpose
/proc/[pid]/stat	Read process CPU times, priority, state
/proc/[pid]/status	Read process memory usage
/proc/stat	Read system-wide CPU times
/proc/meminfo	Read system memory info
std::filesystem	Discover /proc/[pid] directories
getpid() / getppid()	Get process/parent IDs
sysconf()	Get system configuration
clock_gettime()	High-resolution timing

Windows Interfaces

Interface	Purpose
EnumProcesses()	Discover running process IDs
OpenProcess()	Get handle to a process
GetProcessMemoryInfo()	Read process memory usage
GetProcessTimes()	Read process CPU times
QueryFullProcessImageNameA()	Read process name
GlobalMemoryStatusEx()	Read system memory

6 Data Models

ProcessInfo (Real OS Data)

This struct stores actual data collected from the operating system. It contains fields that correspond directly to /proc or WinAPI data.

```
struct ProcessInfo {
    int pid;                // Process ID
    int parentPid;          // Parent process ID
    std::string name;       // Process name
    char state;             // R=Running, S=Sleeping
    long priority;          // Kernel priority
    long niceValue;         // Nice value (-20 to 19)
    unsigned long userTime; // User CPU time (ticks)
    unsigned long kernelTime; // Kernel CPU time (ticks)
    unsigned long startTime; // Process start time
    unsigned long virtualMemory; // Virtual memory size
    long residentMemory;    // Physical memory (RSS)
    int threadCount;        // Number of threads
    double cpuUsage;        // CPU utilization %
};
```

SchedulingProcess (Textbook Model)

This struct represents a process for scheduling analysis. Real OS processes do not have clean 'arrival time' and 'burst time' values, so these are estimated.

```
struct SchedulingProcess {
    int pid;
    std::string name;
    double arrivalTime; // Estimated from start time
    double burstTime;   // Estimated from CPU time
    double remainingTime; // For preemptive algorithms
    int priority;        // From OS priority/nice
    double firstStartTime;
    double completionTime;
    double turnaroundTime; // CT - AT
    double waitingTime;    // TAT - BT
    double responseTime;   // First start - AT
};
```

7 Scheduling Algorithms

7.1 FCFS (First Come First Serve)

Non-preemptive. Processes executed in arrival order. Simple but can cause convoy effect.

- Sort processes by arrival time
- Execute each process to completion
- Handle idle periods when no process has arrived

7.2 SJF (Shortest Job First)

Non-preemptive. Selects shortest burst time. Optimal for minimizing average waiting time.

- At each step, select shortest burst among available
- Handle ties using arrival time
- Skip idle periods

7.3 SRTF (Shortest Remaining Time First)

Preemptive SJF. Minimum remaining execution time. Records preemptions.

- At each time unit, check for newly arrived processes
- Preempt if shorter process arrives
- Track context switches

7.4 Round Robin

Preemptive with configurable quantum (default $Q=2$). Each process gets a time slice.

- Use a FIFO ready queue
- Execute $\min(\text{quantum}, \text{remainingTime})$ per turn
- Smaller quantum = more context switches, better response time

7.5 Priority Scheduling

Non-preemptive. Lower numerical priority = higher priority. Ties broken by arrival time.

- Select highest priority (lowest number) available process
- Handle ties using arrival time

8 Performance Metrics

Metric	Formula	Purpose
Completion	When process finishes	Total execution time
Turnaround	CT - AT	Total time in system
Waiting	TAT - BT	Time spent waiting
Response	First start - AT	Time to first response
CPU Util	Busy/Total x 100	CPU efficiency %
Throughput	Count / Total time	Work completion rate
Fairness	Jain's index (0-1)	Equity of service

Jain's Fairness Index

$\text{Fairness} = (\text{sum})^2 / (n * \text{sum_of_squares})$

Example: waiting = [0, 7, 10, 11]

sum=28, sum_sq=270

Fairness = 784 / 1080 = 0.7259

9 Optimization Engine

The optimization engine compares all five algorithms and recommends the best one based on measurable performance metrics.

Weighting Configuration

Metric	Weight	Direction
Average Waiting Time	25%	Lower=Better
Average Response Time	25%	Lower=Better
Average Turnaround Time	15%	Lower=Better
CPU Utilization	15%	Higher=Better
Fairness	10%	Higher=Better
Context Switch Overhead	10%	Lower=Better

Normalization

Lower-is-better: $\text{score} = \text{bestValue} / \text{currentValue}$
Higher-is-better: $\text{score} = \text{currentValue} / \text{bestValue}$
Protected against division by zero.

Final Score

$\text{Final} = (\text{wait} * \text{ws} + \text{resp} * \text{rs} + \text{turn} * \text{ts} + \text{cpu} * \text{cs} + \text{fair} * \text{fs} + \text{swt} * \text{ss}) * 100$
Result: 0-100 scale. Highest score recommended.
Explanation generated dynamically from metrics.

10 Workload Classification

Type	Condition
Light	< 20 processes
Medium	20-100 processes
Heavy	> 100 processes
CPU-Bound	> 50% high CPU usage
I/O-Bound	> 50% low CPU usage
Interactive	> 40% moderate CPU
Mixed	Active + sleeping

11 Algorithm Test Results

Test processes:

```
P1: Arrival=0, Burst=8, Priority=2
P2: Arrival=1, Burst=4, Priority=1
P3: Arrival=2, Burst=2, Priority=3
P4: Arrival=3, Burst=1, Priority=2
```

Results

Algo	Avg Wait	Avg TAT	Avg Resp	CPU%	Switch
FCFS	7.00	10.75	7.00	100	3
SJF	5.50	9.25	5.50	100	3
SRTF	2.75	6.50	0.25	100	5
RR(Q=2)	5.00	8.75	2.00	100	6
Priority	6.75	10.50	6.75	100	3

Optimization Scores

Algorithm	Score
SRTF	91.8 / 100
Round Robin	58.0 / 100
SJF	57.1 / 100
Priority	53.7 / 100
FCFS	53.1 / 100

Recommended: SRTF (91.8/100)

12 Real System Test Results

Tested on a real Windows machine with 98 active processes.

Results on Real Data

Algo	Avg Wait	Avg TAT	Avg Resp	CPU%	Switch
FCFS	24.15	28.37	24.15	100	14
SJF	18.07	22.30	18.07	100	14
SRTF	18.07	22.30	18.07	100	14
RR(Q=2)	33.87	38.10	10.97	100	40
Priority	24.15	28.37	24.15	100	14

Recommended: SJF (87.2/100)

13 Build Instructions

Prerequisites

- C++17 compiler (GCC, Clang, or MSVC)
- CMake 3.16+

Build Commands

```
cmake -S . -B build
cmake --build build

./build/cpu_scheduler      # Linux
.\build\cpu_scheduler.exe  # Windows

./build/test_runner        # Run tests
```

Build Outputs

File	Description
cpu_scheduler.exe	Main application
test_runner.exe	Test suite (35 tests)
*.dll (x3)	MSYS2 runtime deps

14 Limitations

- /proc reading only works on Linux; Windows uses alternative APIs
- Real OS does not expose textbook burst times directly
- Analyzes workload but does not control the kernel scheduler
- CPU usage accuracy depends on measurement interval
- Short-lived processes may disappear during collection
- Single snapshot approximation on non-Linux systems

15 Viva Preparation - Key OS Concepts

CPU Scheduling

- Why scheduling? CPU is shared; scheduler decides who runs when
- Preemptive vs Non-preemptive: Can OS interrupt a running process?
- Context Switch: Save/restore state when switching processes
- Turnaround Time: Submission to completion
- Waiting Time: Time in ready queue
- Response Time: Submission to first execution

Process Management

- PCB: Kernel data structure storing process state
- Process States: Running, Ready, Blocked, Terminated
- System Calls: Interface between user and kernel
- /proc: Virtual filesystem exposing kernel data

Memory Management

- Virtual Memory: Each process has own address space
- RSS: Physical memory currently used
- Virtual Size: Total address space including swap

Optimization

- Jain's Fairness Index: Equity of resource distribution
- Normalization: Scale metrics to comparable ranges
- Multi-criteria optimization: Weighted scoring